

Which Database Does This Project Use?

PQC Secure Communication - Simple Explanation | Database: SQLite

Short answer: This project uses SQLite as its default database. The Python application uses Flask-SQLAlchemy to work with the database.

SQLite is a relational database that stores data in a local file and does not require a separate database server. This makes the project easy to install and run.

Database type

SQLite - file-based relational database

Database file

pqc_chat.db

Python connection

Flask-SQLAlchemy ORM

Default location

The application's instance folder

Simple example: Think of SQLite as a digital notebook. The application records users, messages, emails and other information in separate tables.

What Does the Database Do in This Project?

When the application starts, ``db.create_all()`` creates the required tables. Information from registration, chat, email and file upload activities is stored in these tables.

Table	Purpose	Main information stored
users	User account and identity	username, email, password hash, cryptographic keys
messages	Chat messages	encrypted message, IV, authentication tag, signature
emails	Secure email	subject, encrypted body, sender/receiver, signature
attachments	Email/chat files	filename, file size, encrypted file key, SHA3 digest
group_chats	Group chat	group name, admin, creation time
group_members	Group membership	Which users belong to which groups
audit_logs	Security activity tracking	action, algorithm, result, risk level, IP, time
benchmark_results	Performance measurement	Operation time and data size
user_session_keys	Chat session status	session hash, mode, ACTIVE/REVOKED status
user_chat_preferences	Chat settings	Pinned, archived, blocked and locked settings

How Is the Data Protected?

The key design point is that chat messages and email bodies are not stored as plain text. The database stores the ciphertext created after encryption.

- Password: The original password is not stored; only the password hash is stored.
- Chat message: encrypted_payload, IV, authentication tag and digital signature are stored.
- Email: encrypted_body, sender/receiver details and signature information are stored.
- File: The encrypted file is kept in server storage; the database keeps file metadata, wrapped key and SHA3 digest.
- Audit log: Login events, security actions, success/failure and risk level history are stored.

User action	Application	Database
User sends a message	Encrypts and signs it	Stores only ciphertext
Receiver opens the message	Verifies and decrypts it	Reads the stored encrypted data

Important Note

The database stores encrypted data. However, this current prototype stores private keys in database fields. Before production deployment, add encryption at rest, strong secret management and proper access control.

Can the Database Setting Be Changed?

Yes. This setting is defined in `app/config.py`:

```
SQLALCHEMY_DATABASE_URI = os.environ.get('DATABASE_URL') or 'sqlite:///pqc_chat.db'
```

What this means:

- If the `DATABASE\_URL` environment variable is not provided, SQLite is used.
- If `DATABASE\_URL` is provided, SQLAlchemy uses that database connection.
- Tests use `sqlite:///memory:` separately. This is a temporary test database, not the main application database.

Final answer

This project's default database is SQLite, accessed from Python through Flask-SQLAlchemy. User accounts, encrypted chat, secure email, file information, groups and security audit records are stored in relational tables. Important message and email content is stored in encrypted form rather than plain text.

Source files: `app/config.py`, `app/models.py`, `app/__init__.py`